



Homework Assignment

Week 10: 2D Lists and Matrix Operations

Total Points: 100 — Due: Next Class



Name: _____

Date: _____

Important Instructions

- Work on your own, but you may ask family members for hints only if stuck
- Write your code in Python (*using Thonny or any online IDE*)
- For setting up your code files, kindly see `instructions.pdf`
- For written answers, use the spaces provided on this sheet
- Show your work for full credit!

1 Part 1: Understanding 2D List Structure (15 points)

Time estimate: 20 minutes

1.1 Exercise 1.1: Memory Diagram - The Right Way (8 points)

Draw a memory diagram for this correctly-created 2D list:

```
1 grid = [[1, 2, 3] for i in range(2)]
```

Your memory diagram:

Instructions:

- Draw a box for the variable `grid`
- Draw the outer list with 2 elements (references)
- Draw two SEPARATE inner lists, each containing `[1, 2, 3]`

- Use arrows to show references from outer list to inner lists
- Label each list with its memory ID (you can use fake IDs like 0x1000, 0x2000)

1.2 Exercise 1.2: Memory Diagram - The Wrong Way (7 points)

Now draw a memory diagram for this INCORRECTLY-created 2D list:

```
1 bad_grid = [[1, 2, 3] * 1] * 2
```

Your memory diagram:

Key question: How many inner list objects exist in memory? _____

Explain why this is a problem:

2 Part 2: Creating 2D Lists Correctly (18 points)

Time estimate: 25 minutes

2.1 Exercise 2.1: Pattern Recognition (10 points)

For each desired 2D list structure, write the correct list comprehension to create it:

1. Create a 3x4 grid filled with zeros (3 rows, 4 columns):

```
1 grid1 = _____
```

2. Create a 5x5 grid filled with the value 'X':

```
1 grid2 = _____
```

3. Create a 2x3 grid where each element is its row number (0-indexed):

```
1 # Should produce: [[0, 0, 0], [1, 1, 1]]
2 grid3 = _____
```

2.2 Exercise 2.2: Multiple Choice - Common Mistakes (8 points)

Circle the correct answer:

1. Which creates independent rows for a 3x3 grid?

- A. `grid = [[0] * 3] * 3`

- B. `grid = [[0 for j in range(3)] for i in range(3)]`
- C. `grid = [0] * 9`
- D. `grid = [[0, 0, 0]] * 3`

2. After this code, what happens?

```
1 matrix = [[0] * 2] * 3
2 matrix[0][0] = 1
```

- A. Only `matrix[0][0]` becomes 1
- B. All first elements of each row become 1
- C. The entire matrix becomes 1s
- D. Python shows an error

3. What's the correct way to access the element at row 2, column 1?

- A. `grid[1][2]`
- B. `grid[2][1]`
- C. `grid(2, 1)`
- D. `grid[2, 1]`

4. How many list objects are created by this code?

```
1 board = [[' ' for j in range(3)] for i in range(3)]
```

- A. 1 (just the outer list)
- B. 3 (just the rows)
- C. 4 (1 outer + 3 inner lists)
- D. 9 (one for each element)

3 Part 3: Accessing and Modifying 2D Lists (20 points)

Time estimate: 25 minutes

3.1 Exercise 3.1: Code Tracing and Prediction (12 points)

What will this program print? Trace through each step carefully:

```
1 matrix = [[1, 2, 3],
2           [4, 5, 6],
3           [7, 8, 9]]
4
5 print(matrix[1][1])           # Line 1
6 print(matrix[0][2])           # Line 2
7 print(matrix[2])               # Line 3
8
9 matrix[1][1] = 0
10 print(matrix[1])              # Line 4
```

Your predictions:

Line 1 output: _____

Line 2 output: _____

Line 3 output: _____

Line 4 output: _____

3.2 Exercise 3.2: Code Completion - Grid Access (8 points)

Open [week10_ex3.2.py](#) and complete this program:

```

1  # Grid Access Practice
2  # Create a 4x3 seating chart
3  seating = [['Ali', 'Sara', 'Ahmed'],
4             ['Fatima', 'Hassan', 'Zainab'],
5             ['Omar', 'Ayesha', 'Bilal'],
6             ['Maryam', 'Ibrahim', 'Hina']]
7
8  print("== Classroom Seating Chart ==")
9
10 # Print student in row 0, seat 1
11 print(f"Front row, middle seat: {_____}")
12
13 # Print student in row 2, seat 0
14 print(f"Third row, first seat: {_____}")
15
16 # Change Hassan to Hamza (he moved!)
17 seating_____
18
19 # Print entire second row
20 print(f"Second row students: {_____}")
21
22 # Count total students
23 total = 0
24 for row in seating:
25     total += _____
26
27 print(f"Total students: {total}")

```

4 Part 4: Traversing 2D Lists with Nested Loops (22 points)

Time estimate: 30 minutes

4.1 Exercise 4.1: Complete the Patterns (12 points)

Open [week10_ex4.1.py](#) and complete these nested loop patterns:

```

1  # Pattern 1: Sum all elements in a 2D list
2  numbers = [[1, 2, 3],
3             [4, 5, 6],
4             [7, 8, 9]]
5
6  total = 0
7  for row in numbers:

```

```

8     for element in _____:
9         total += _____
10
11 print(f"Sum of all elements: {total}")
12
13 # Pattern 2: Find maximum value in entire grid
14 grid = [[15, 23, 8],
15         [42, 19, 31],
16         [7, 36, 12]]
17
18 max_val = grid[0][0] # Start with first element
19 for i in range(len(grid)):
20     for j in range(len(grid[i])):
21         if _____ > max_val:
22             max_val = _____
23
24 print(f"Maximum value: {max_val}")
25
26 # Pattern 3: Count zeros in a grid
27 matrix = [[0, 1, 0],
28           [1, 0, 1],
29           [0, 0, 1]]
30
31 zero_count = 0
32 for _____ in range(len(matrix)):
33     for _____ in range(len(matrix[row])):
34         if matrix[row][col] == 0:
35             zero_count += 1
36
37 print(f"Number of zeros: {zero_count}")

```

4.2 Exercise 4.2: Display Grid Nicely (10 points)

Open [week10_ex4.2.py](#) and complete this grid display function:

```

1 # Display a 2D list as a formatted grid
2 def display_grid(grid):
3     """Display a 2D grid with proper formatting"""
4     print("Grid:")
5     for row in grid:
6         # Print each element with spacing
7         for element in row:
8             print(f"{element:4}", end="") # 4 spaces width
9             print() # New line after each row
10
11 # Test with number grid
12 number_grid = [[1, 2, 3, 4],
13               [5, 6, 7, 8],
14               [9, 10, 11, 12]]
15
16 display_grid(number_grid)
17
18 # Now create a multiplication table (3x3)
19 mult_table = [[__ for j in range(1, 4)] for i in range(1, 4)]
20
21 print("\nMultiplication Table (1-3):")
22 display_grid(mult_table)
23
24 # Challenge: Fill in the multiplication calculation

```

25 *# Hint: Each cell should be (row number) * (column number)*

5 Part 5: Debugging 2D List Programs (15 points)

Time estimate: 20 minutes

Each snippet has bugs related to 2D lists. Find and fix ALL errors:

5.1 Snippet 1: The Wrong Way Bug (8 points)

```
1 # This code tries to create a tic-tac-toe board
2 # But it has the aliasing bug!
3
4 board = [[' ' * 3] * 3]
5
6 # Player X makes a move in top-left
7 board[0][0] = 'X'
8
9 # Display board
10 for row in board:
11     print(row)
12
13 # BUG: X appears in all rows, not just the first!
```

What's wrong? _____

Fixed version:

1 *# Write the corrected code here:*

Save your corrected code as week10-ex5-1-fixed.py

5.2 Snippet 2: Index Confusion (7 points)

```
1 # This code tries to sum each row
2 # But the indices are wrong
3
4 scores = [[85, 90, 78],
5           [92, 88, 91],
6           [75, 82, 89]]
7
8 print("Row totals:")
9 for i in range(len(scores)):
10     row_total = 0
11     for j in range(len(scores)): # BUG HERE!
12         row_total += scores[j][i] # BUG HERE TOO!
13     print(f"Row {i}: {row_total}")
```

What's wrong? _____

Fixed version:

1 *# Write the corrected code here:*

Save your corrected code as `week10_ex5_2_fixed.py`

6 Part 6: Full Program - Tic-Tac-Toe Board (25 points)

Time estimate: 40 minutes

6.1 Exercise 6.1: Build a Tic-Tac-Toe Game Board (25 points)

Create a program called `tictactoe_board.py` that manages a tic-tac-toe game:

Program Requirements

Your program should:

- Create a 3x3 board using the CORRECT method (list comprehension)
- Display the board in a nice format with grid lines
- Allow players to make moves by entering row and column
- Validate moves (check if position is empty)
- Alternate between 'X' and 'O' players
- Check if a position is already taken
- Display the board after each move
- Continue for at least 5 moves (no win detection needed yet)
- Show clear messages for invalid moves

Example Run:

```
=== Tic-Tac-Toe Game ===
Creating board correctly with list comprehension...
```

Current Board:

```
| |
-+-+
| |
-+-+
| |
```

Player X's turn

Enter row (0-2): 1

Enter column (0-2): 1

Current Board:

```
| |
-+-+
|X|
-+-+
| |
```

Player O's turn

```
Enter row (0-2): 1
Enter column (0-2): 1
That position is already taken! Try again.
```

```
Player 0's turn
Enter row (0-2): 0
Enter column (0-2): 0
```

Current Board:

```
0| |
-+-+
|X|
-+-+
| |
```

Starter Structure:

```
1  # tictactoe_board.py
2  # Name: -----
3  # Date: -----
4
5  print("=== Tic-Tac-Toe Game ===")
6  print("Creating board correctly with list comprehension...")
7
8  # Create 3x3 board - USE CORRECT METHOD!
9  board = [[' ' for j in range(3)] for i in range(3)]
10
11 def display_board(board):
12     """Display the board with grid lines"""
13     # Your code to display board nicely
14     # Hint: Use nested loops and print separators
15     pass
16
17 def is_valid_move(board, row, col):
18     """Check if a move is valid"""
19     # Check if row and col are in range
20     # Check if position is empty
21     # Return True or False
22     pass
23
24 def make_move(board, row, col, player):
25     """Place player's mark on the board"""
26     # Update the board at [row][col] with player ('X' or 'O')
27     pass
28
29 # Main game loop
30 current_player = 'X'
31 moves_made = 0
32
33 while moves_made < 5: # Play 5 moves
34     display_board(board)
35     print(f"\nPlayer {current_player}'s turn")
36
37     # Get move from player
38     row = int(input("Enter row (0-2): "))
39     col = int(input("Enter column (0-2): "))
40
41     # Validate and make move
42     if is_valid_move(board, row, col):
```



```

43     make_move(board, row, col, current_player)
44     moves_made += 1
45
46     # Switch players
47     if current_player == 'X':
48         current_player = 'O'
49     else:
50         current_player = 'X'
51 else:
52     print("That position is already taken! Try again.")
53
54 # Final board
55 print("\nFinal Board:")
56 display_board(board)
57 print("\nGame demonstration complete!")

```

Testing Checklist:

- ☐ Board is created with CORRECT list comprehension (no aliasing!)
- ☐ Board displays with proper grid formatting
- ☐ Program validates row and column inputs
- ☐ Program checks if position is already taken
- ☐ Players alternate between X and O correctly
- ☐ Board updates correctly after each move
- ☐ Program handles at least 5 moves
- ☐ No crashes on invalid positions

7 Part 7: Reflection Questions (5 points)

Time estimate: 10 minutes

Answer these questions in complete sentences:

1. Explain why `[[0]*3]*3` doesn't create independent rows. Use the word "reference" in your answer.

2. When accessing a 2D list with `grid[i][j]`, which comes first - row or column? Why is this important to remember?

3. What real-world data could you represent with a 2D list? Give at least two examples besides game boards.

8 Bonus Section: Extra Challenges (Optional - 10 extra points)

Only attempt if you've finished everything else!

8.1 Challenge 1: Matrix Operations (5 points)

Open `week10_bonus1.py` and implement these matrix operations:

```

1  # Matrix Operations Challenge
2
3  def transpose_matrix(matrix):
4      """Return the transpose of a matrix (swap rows and columns)
5      Example: [[1,2],[3,4]] becomes [[1,3],[2,4]]
6      """
7      # Your code here
8      pass
9
10 def add_matrices(matrix1, matrix2):
11     """Add two matrices element by element
12     Both matrices must be same size
13     """
14     # Your code here
15     pass
16
17 # Test your functions
18 m1 = [[1, 2, 3],
19       [4, 5, 6]]
20
21 print("Original matrix:")
22 for row in m1:
23     print(row)
24
25 print("\nTransposed matrix:")
26 transposed = transpose_matrix(m1)
27 for row in transposed:
28     print(row)
29
30 m2 = [[10, 20],
31       [30, 40]]
32
33 m3 = [[1, 2],
34       [3, 4]]
35
36 print("\nMatrix addition:")
37 result = add_matrices(m2, m3)
38 for row in result:
39     print(row)

```

8.2 Challenge 2: Grid Pattern Generator (5 points)

Create a program called `pattern_generator.py` that:

- Creates a 2D grid of specified size (user input)
- Fills it with a pattern (checkerboard, border, diagonal, etc.)
- Displays the pattern nicely

- Allows user to choose different patterns

Example patterns:

Checkerboard (4x4):	Border (5x5):
X 0 X 0	X X X X X
0 X 0 X	X 0 0 0 X
X 0 X 0	X 0 0 0 X
0 X 0 X	X 0 0 0 X
	X X X X X

9 Debugging Discoveries (5 bonus points)

Growth Mindset: Errors are learning opportunities!

Did you encounter any interesting errors while completing this homework? Share one error and how you fixed it:

🔧 My Debugging Story

The Error I Got: _____

What I Tried First: _____

How I Fixed It: _____

What I Learned: _____

No errors? Share a tricky part you figured out instead!

Submission Checklist

Before submitting, make sure you have:

- ☐ Completed all required sections (Parts 1-7)
- ☐ Drawn memory diagrams for Part 1 (both correct and wrong methods)
- ☐ Saved each code exercise in its own file:
 - week10_ex3_2.py
 - week10_ex4_1.py
 - week10_ex4_2.py
 - week10_ex5_1_fixed.py
 - week10_ex5_2_fixed.py
 - tictactoe_board.py
- ☐ Created a folder named week10_hw_yourname with all files

- ☐ Tested each code file individually to ensure it runs without errors
- ☐ Verified that your 2D lists use the CORRECT creation method
- ☐ Checked that nested loops use proper row/column indexing
- ☐ Written your name at the top of this paper
- ☐ Answered all reflection questions
- ☐ Attempted bonus section (optional)

Module 2 Concepts Check:

Which concept was most challenging? _____

Do you understand why `[[0]*n]*m` is wrong? _____

Time Tracking:

How long did this homework take you? _____ hours

Parent/Guardian Signature: _____

(Confirming student completed work independently)

Great job mastering 2D lists and matrix operations!
Next week: Dictionaries - organizing data with key-value pairs!